

Orquestación de servicios con Docker Compose

1. Descripción de la actividad

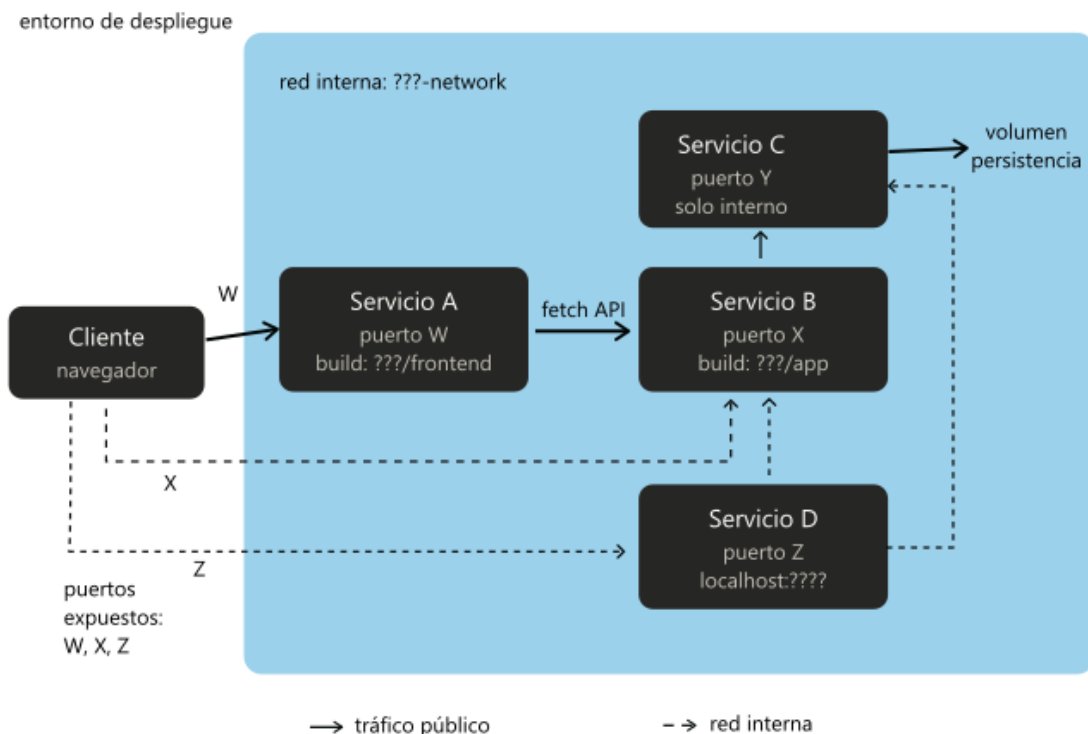
En esta práctica debes construir un stack de aplicaciones completamente funcional usando docker compose. El objetivo es demostrar las ventajas de la orquestación de múltiples contenedores frente al manejo manual con comandos docker run individuales.

El sistema para implementar debe estar compuesto por cuatro servicios que se comunican entre sí a través de una red interna de Docker:

- PostgreSQL — base de datos relacional con datos persistentes en un volumen
- Flask (Python) — API REST con operaciones CRUD sobre la base de datos
- pgAdmin — panel de administración de la base de datos
- Nginx — servidor que sirve un frontend HTML/JS consumiendo la API

2. Arquitectura esperada

La siguiente figura muestra la arquitectura objetivo que se debe reproducir:



Consideraciones de red que se deben respetar:

- PostgreSQL NO debe tener puertos expuestos al host en producción — solo accesible dentro de la red interna que haya creado.
- Flask debe exponer el puerto 3000 al host para permitir pruebas directas con un navegador o curl.
- pgAdmin debe ser accesible desde el navegador en el puerto 8080.
- El frontend debe ser accesible en el puerto 5000.
- Todos los servicios deben pertenecer a la misma red interna.

3. Requisitos técnicos obligatorios

3.1 Archivo docker-compose.yml

- Definir los cuatro servicios: db, web, pgadmin, frontend.
- Usar variables de entorno desde un archivo .env.
- El servicio web debe construirse desde un Dockerfile propio.
- El servicio db debe tener un healthcheck que verifique que PostgreSQL acepta conexiones en la base de datos correcta.
- Los servicios web y pgadmin deben usar depends_on con condition: service_healthy para garantizar que la base de datos esté lista antes de arrancar.
- Definir al menos un volumen nombrado para persistir los datos del contenedor que lo requiera estrictamente.
- Definir el nombre de una red explícitamente.

3.2 Base de datos

- Usar un script SQL en init-db/ que se ejecute automáticamente al inicializar el contenedor.
- El script debe crear al menos tres tablas relacionadas entre sí mediante claves foráneas.
- El script debe incluir datos de ejemplo (mínimo 5 registros por tabla principal).
- NO debe contener instrucciones CREATE DATABASE — PostgreSQL la debe crear por variable de entorno.

3.3 API Flask

- Implementar endpoints CRUD completos (GET, POST, PUT/PATCH, DELETE) para al menos tres entidades.
- Las rutas deben devolver respuestas en formato JSON.
- Debe incluir una ruta raíz GET / que liste los endpoints disponibles.
- Debe tener CORS habilitado para permitir que el frontend consuma la API.

3.4 Frontend

- Página HTML servida por Nginx dentro de un contenedor Docker.
- Consumir al menos tres endpoints de la API.

- Mostrar los datos en una tabla o lista visible en el navegador.
- Incluir un formulario funcional para crear o editar al menos un recurso.

4. Entregables

Debes entregar un repositorio o carpeta comprimida (.zip) con la siguiente estructura:

Archivo / carpeta	Descripción
docker-compose.yml	Archivo principal de orquestación
.env	Variables de entorno (Tener presente buenas prácticas de seguridad)
app/Dockerfile	Dockerfile del servicio Flask
app/app.py	Código de la API REST
app/requirements.txt	Dependencias de Python
app/init-db/01-init.sql	Script SQL de inicialización
frontend/Dockerfile	Dockerfile del servicio Nginx
frontend/index.html	Interfaz de usuario
README.md	Instrucciones de uso y capturas de pantalla

El README.md debe incluir capturas de pantalla que evidencien:

- El comando docker-compose ps mostrando todos los servicios en estado running o healthy.
- La API respondiendo desde el navegador o curl.
- El frontend funcionando en el navegador.
- pgAdmin conectado y mostrando las tablas de la base de datos.

5. Rúbrica de evaluación

Calificación máxima: 5.0 puntos

Criterio	Descripción	Puntos
1. docker-compose.yml	Define los 4 servicios correctamente. Usa variables de entorno desde .env. Tiene healthcheck funcional en db.	1.5

	Usa depends_on con condition: service_healthy. Define red y volumen nombrado.	
2. Base de datos	El script SQL inicializa correctamente sin CREATE DATABASE. Crea mínimo 3 tablas con relaciones. Incluye datos de ejemplo. Los datos persisten al reiniciar con docker-compose down / up.	0.75
3. API Flask	Implementa CRUD completo para al menos 2 entidades. Responde en JSON. Tiene CORS habilitado. Incluye ruta raíz GET / descriptiva. No usa time.sleep() para esperar la DB.	1.0
4. Frontend	Servido desde un contenedor Nginx. Consume la API con fetch(). Muestra datos en tabla o lista. Permite crear o editar al menos un recurso desde la interfaz.	0.75
5. Documentación	README.md con instrucciones claras de uso. Incluye las 4 capturas de pantalla requeridas. La estructura del proyecto coincide con la tabla de entregables.	0.5
6. Buenas prácticas	Credenciales solo en .env, nunca en el código. Contenedor de DB sin puerto expuesto al host. Nombres de contenedores explícitos con container_name. El stack levanta completo con un solo docker-compose up --build.	0.5
TOTAL		5.0

6. Penalizaciones

- Credenciales hardcodeadas en docker-compose.yml o app.py: -0.5 puntos.
- El stack no levanta con docker-compose up --build sin modificaciones: -1.0 punto.
- Ausencia de capturas de pantalla en el README: -0.3 puntos.
- Puerto de PostgreSQL expuesto al host (5432): -0.2 puntos.

7. Comandos de referencia

Para construir y levantar todo el stack:

```
docker-compose up --build
```

Para verificar el estado de los servicios:

```
docker-compose ps
```

Para detener sin eliminar volúmenes:

```
docker-compose down
```

Para reiniciar desde cero (elimina datos):

```
docker-compose down -v
```

```
docker-compose up --build
```

Para ver logs de un servicio específico:

```
docker-compose logs -f web
```

Nota: La entrega es para el miércoles 7 de octubre hasta las 12:00 m. El nombre del archivo (comprimido zip o rar) debe ser su nombre con las iniciales en mayúscula y sin espacios. Debe enviarse por correo electrónico a avargase@unal.edu.co (no debe ser un documento compartido a través de drive) y en el asunto del correo: *Actividad1_Tendencias*.

Sí dado el caso, el servicio de correo no le permite adjuntar el archivo, puede crear un repositorio en GitHub y enviarme la invitación a aldemirvargas@hotmail.com